Mp10 — Signature Helper

Signature capture and document scanning at the workstation, for Mp10 Web

Structured Systems · www.structuredsystems.com

20 August 2026

Contents

1 About this guide **3**
- 1.1 The short version . . . 3
- 1.2 Why there is a separate program at all . . . 3

2 Using it — front desk **5**
- 2.1 Scanning, at the same desk . . . 5
- 2.2 What it does not do . . . 5

3 When something is wrong **6**
- 3.1 The signature pad . . . 6
- 3.2 Scanning . . . 6

4 Installing it — IT task **8**
- 4.1 It is not a service . . . 8
- 4.2 The quick way: the workstation bundle . . . 8
- 4.3 What to copy . . . 8
- 4.4 Settings . . . 9
- 4.5 Telling MpSigSrv which pad . . . 9
- 4.6 The consent text . . . 10
- 4.7 Check it . . . 11
- 4.8 Updating a workstation later . . . 12

5 Scanning — IT task **13**
- 5.1 What a scanner needs on the workstation . . . 13
- 5.2 WIA and TWAIN, and why both . . . 13
- 5.3 How the helper decides what a scanner can do . . . 13
- 5.4 The cache, and Re-detect . . . 13
- 5.5 A scanner that owns its own settings . . . 14
- 5.6 What a scanned page becomes . . . 14
- 5.7 Cropping a card . . . 14
- 5.8 When a scanner is not found . . . 15
- 5.9 What has never been tested . . . 15

6 Notes for the curious **16**
- 6.1 Is it safe to have a program listening on the workstation? . . . 16
- 6.2 Why does the operator code travel with each capture? . . . 16
- 6.3 What happens if a capture hangs . . . 16
- 6.4 Why a scan makes the desk feel slow . . . 16
- 6.5 Where the PDF comes from . . . 16
- 6.6 Where scanned pages live while a scan is in progress . . . 16
- 6.7 Does an https site really reach a http://127.0.0.1 helper? . . . 17

1 About this guide

Some of what Mp10 Web does happens on **the computer the operator is sitting at**, not on the server: taking a signature on a pad, and scanning a document. Both need hardware plugged into that desk, and a browser cannot reach it. A small companion program called **MpSigSrv** does those jobs on the workstation's behalf.

MpSigSrv does two things, and the name only mentions one of them. It was written for the Topaz signature pad and kept its name when scanning was added to it. If you are here because a *Scan* button does not work, you are in the right guide.

It does	Which is	Covered in
Patient signatures on a **Topaz pad**	the original job	*Installing it*
Scanning a document or a card	added later, same program	*Scanning*

Neither half needs the other. A desk with a scanner and no pad is a perfectly normal installation, and so is the reverse — you install the one program and attach whatever that desk has.

There are two audiences here. **Using it** is for the person at the front desk and is two pages. Everything from *Installing it* onward is an **IT task** and is where the setup actually lives.

If you have used **MpPrintSrv**, the print helper, this will look familiar — same idea, same loopback design. **One thing is different and it matters: MpPrintSrv is a Windows service; MpSigSrv is not and cannot be.** See It is not a service.

> **A note on paths.** This guide says *the Mp10 folder* for wherever the Mp10 programs are installed on that workstation. It is `C:\Mp10` on a typical installation, but it is a per-site choice — check where `Patients10.exe` lives on the machine in front of you and use that.

1.1 The short version

Program	`MpSigSrv.exe`, in the Mp10 program folder (`C:\Mp10` on a typical install)
Runs on	the workstation with the pad or the scanner, in the signed-in user's session
Started by	`MpSigSrv.exe -listen`, at logon
Port	**6266** (MpPrintSrv is 6265; they coexist)
Is it running?	open http://127.0.0.1:6266/ping in the browser on that PC
What can it scan?	open http://127.0.0.1:6266/scanners on that PC
Settings	`[RDD]` and `[SIGN]` in `Mp10.ini`, beside the exe
Written by it	`MpScanners.ini`, beside the exe — what this desk's scanners can do
Logs	`MpSigSrv.log` and `Topaz.log`, beside the exe

1.2 Why there is a separate program at all

The pad faces the **patient**. The consent text, the choice they make and the Next control all belong on the pad's own small screen, exactly as the Mp10 desktop puts them there — the browser is the *operator's* screen, and the patient never looks at it.

A browser cannot drive that screen. Beyond that, the consent text is allowed to contain expressions the Mp10 desktop evaluates at the moment of signing (`^email` is the one this practice uses, and `~…~` can hold

any Harbour expression). Reproducing that in JavaScript is not merely awkward, it is not possible. So MpSigSrv runs **the desktop's own signing code**, and a patient signing from the web sees exactly what a patient signing from the desktop sees.

MpSigSrv also **stores the signature itself**, through the same statement the desktop uses. Nothing about the signature travels back through the browser.

Scanning is here for a simpler reason: **the scanner is plugged into the desk, not into the server**, and a web page cannot talk to a scanner at all. Windows offers two ways to drive one — WIA and TWAIN — and neither is reachable from JavaScript. So the same program that already runs at that desk, and is already trusted by that browser, does the scanning too. Adding it to MpSigSrv rather than shipping a third program means one install, one startup entry and one setting to get wrong instead of three.

---

2 Using it — front desk

1. Open the encounter in Mp10 Web and press **Sign**.
2. The browser shows you the consent the patient is being shown — this is your copy to follow, not theirs.
3. Press **Capture on pad**.
4. **Hand the pad to the patient.** They read the text, page through it with the arrow, tap their choice if the consent offers one, and sign.
5. A window appears on your screen showing the signature they gave. Press the pen button to keep it, or Cancel to discard it and start again.
6. The browser says *"Signature captured on the pad and saved."*

The options shown in the browser are greyed out on purpose. **The patient chooses on the pad**; a choice made on your screen would be a consent they never gave.

2.1 Scanning, at the same desk

The same program answers the **Scan** buttons in Mp10 Web. There are two of them and they behave differently on purpose:

- **Scan...** in the patient's *Attachments* window scans a document, a page at a time. Several pages become one PDF; a single page stays an image.
- **Scan** beside the patient ID card and the insurance card scans **one page and crops it to the card**, the way the Mp10 desktop already does.

The step-by-step is in the **Mp10 Web User Guide** under *Capturing images* and *Attachments*, because it is a desk task rather than an IT one. What belongs here is the part that sends people looking for help: **if the Scan button is greyed out, this program is not running at that desk** — the same cause as a pad that will not wake up, and the same fix.

2.2 What it does not do

- **Patient-record (HIPAA) signatures** are not on the pad from the web yet. Only encounter signatures are. The desktop still does the patient record.
- **It is Windows-only**, because the pad and the scanner drivers are. A Mac or a tablet cannot capture from a pad or scan at all — attach the signature or the image from the record instead, or photograph the card with the device's own camera.
- **It does not scan both sides of a card.** These fields hold one image, and a scan replaces what is there, exactly as *Replace* does.

---

3 When something is wrong

3.1 The signature pad

What you see	What it means	What to do
The signature helper is not answering on this workstation	Either MpSigSrv is not running here, **or** its `[SIGN] ORIGIN` does not list this site. A browser cannot tell those apart.	IT task — see Check it. Check `ORIGIN` **first**; it is the more common of the two.
No operator code for the signed-in user	This Mp10 Web account has no Operator set. The signature would not record who witnessed it, so it is refused.	Set an Operator on that account in the desktop **Admin** → **Users**.
The consent text for this signature cannot be read	`sys_registry` points at a consent file the server cannot open.	IT task — see The consent text.
The signature pad could not be activated	MpSigSrv could not talk to the pad.	Check the pad is plugged in and `[Tablet] TabletComPort` in `C:\Windows\SigPlus.ini` matches its COM port.
The pad shows nothing, or the wrong size text	The pad geometry is wrong.	See Telling MpSigSrv which pad.
The capture did not finish within 300 seconds	Nobody completed the signature and the helper gave up.	Press **Capture on pad** again. If it keeps happening, restart MpSigSrv.

3.2 Scanning

What you see	What it means	What to do
The **Scan** button is greyed out	MpSigSrv is not answering at this desk — not running, or its `ORIGIN` does not list this site.	The same fix as a pad that will not start: Check it.
The signature helper is running on this workstation, but it can see no scanner	The helper answered; Windows reports no scanner.	Switch the scanner on, connect it, press **look again**. If it is on and connected, see When a scanner is not found.
The admin page says *this build predates scanning*	The helper at this desk is an older build. It has no scanning in it at all.	Copy the current `MpSigSrv.exe` to that desk — Updating a workstation later. Nothing else is wrong.
A message naming the driver, after several seconds	The scanner itself refused or failed — paper jam, lid, cable, driver.	The text is the driver's own. Fix what it names and scan again.
That scan is N MB, over the 4 MB limit	Too many pages in one document.	Attach what you have and scan the rest as a second document, or choose a lower resolution.
The scan works but the **card is not cropped**	The crop found nothing darker than the background, so it kept the whole page rather than fail.	Usually a **dark or open scanner lid**. Close the lid. An uncropped card is still a usable card.

What you see	What it means	What to do
Scanning stops after one page from the feeder	The device reported no more paper.	See What has never been tested — the feeder is the least-proven path here.
Everything is slow, and `/ping` does not answer during a scan	Normal. A page takes several seconds and the helper does one thing at a time.	Wait.

Two log files sit beside the exe and between them they answer most of this:

- **`MpSigSrv.log`** — what the helper did: what it connected to, whether it is listening, each capture requested, the option picked, each scanner probe and scan, and every refusal.
- **`Topaz.log`** — what the *pad* reported: its model number and its screen size. This is the file to read when the layout looks wrong.

A third file, **`MpScanners.ini`**, is not a log but is worth reading for the same reason: it is what the helper believes each scanner at this desk can do. See The cache, and Re-detect.

---

4 Installing it — IT task

4.1 It is not a service

MpSigSrv cannot be installed as a Windows service, and there is no `-i` switch. It hosts the Topaz ActiveX control and opens a window; a service runs in session 0, which has no interactive desktop, so neither would work.

It runs **in the signed-in operator's own session**, started at logon. That is a real difference from MpPrintSrv, which is a service — if you set MpSigSrv up the way you set that up, it will not work.

Practically, that means one of:

- a shortcut to `MpSigSrv.exe -listen` in the user's **Startup** folder (`shell:startup`), or
- **Task Scheduler**, trigger *At log on*, action `<Mp10 folder>\MpSigSrv.exe`, arguments `-listen`, and **"Run only when user is logged on"** — not the "whether or not" option, which puts it back in session 0.

It shows no window while it waits. To stop it, end `MpSigSrv.exe` in Task Manager.

4.2 The quick way: the workstation bundle

The same installer that sets up the print helper installs this one too — it copies `MpSigSrv.exe` and `EZTW32.DLL`, and writes `[SIGN] PORT` and `[SIGN] ORIGIN` into `Mp10.ini`, editing only those lines. The one thing it may add is a commented-out `[RDD]` template, and only when that section is missing entirely. See *The quick way* in **Mp10 Web — Printing**.

Two parts of this helper it cannot do for you, and says so rather than pretending otherwise:

- **Starting it at logon.** MpSigSrv runs in the operator's own session, and a Startup shortcut created by a technician lands in the *technician's* profile — where it silently never runs for the person who needs it. The installer prints the exact shortcut and Task Scheduler settings instead; the exact shortcut and Task Scheduler settings are under It is not a service above.
- **Installing SigPlus.** Topaz's control is COM-registered by Topaz's own installer. The workstation installer checks the registration and reports whether signature capture will work; scanning is unaffected either way.

4.3 What to copy

Onto each workstation that has a pad:

File	What it is	Needed for
`MpSigSrv.exe`	the helper	everything
`Mp10.ini`	settings — copy `Mp10.ini.example` and edit	everything
`ace32.dll`, `axcws32.dll`	the **32-bit** Advantage client, already present on any PC running the Mp10 desktop apps	everything
`SigPlus.ocx`	Topaz's control — installed and **registered** by Topaz's own SigPlus installer, not copied by hand	the pad
`EZTW32.DLL`	the TWAIN layer, from the Mp10 desktop install set	TWAIN scanners only

MpSigSrv is a 32-bit program and uses the **same 32-bit ACE client the desktop applications use**. This is the opposite of Mp10 Web's server, which needs the 64-bit client — do not confuse the two.

Nothing in that table can stop the helper from starting except the first three. `SigPlus.ocx` and `EZTW32.DLL` are both looked up when they are first needed rather than when the program loads, so a desk with no pad, or no TWAIN scanner, or neither, still starts and still does the half it can. This is deliberate and it was learned the hard way: an earlier build required `EZTW32.DLL` to *load*, and on any machine without that file the exe died at startup with `0xC0000135` — which meant a missing scanner DLL took the signature pad down with it. **If a build ever behaves that way again, the answer is the file, not the program.**

`gdiplus.dll` is used for every scanned page and is a Windows component present since XP. There is nothing to install and nothing to copy.

`SigPlus.ocx` comes from Topaz. Install **SigPlus** (not SigWeb — that is the browser component and is not used here) from https://www.topazsystems.com, which registers the control and writes `C:\Windows\SigPlus.ini`.

4.4 Settings

`Mp10.ini`, beside the exe. Two sections matter.

```
[RDD]
RDD-VERSION=REMOTE
PATH=\\adsserver\adsdata\sfi\
user=AutoTasks
password=mp10

[SIGN]
PORT=6266
ORIGIN=https://mp10web.example.org
```

`PORT` — 6266 unless something else on the workstation wants it. It is also compiled into Mp10 Web as the default, so changing it here means changing it there too (`VITE_SIGN_URL`). MpPrintSrv's 6265 is a different program on a different port; both can run at once.

`ORIGIN` — the address staff open Mp10 Web at, exactly as the browser sends it: scheme and host, no trailing slash, and the port when it is not 80 or 443. Several may be listed, comma separated.

> **This is the whole of the access control, and it is the thing most likely to be wrong.** With nothing here, every browser request is refused — and a browser cannot tell a refused origin from a helper that is not running. Both arrive as the same failure. If Mp10 Web says the helper is not answering, check this line before anything else. `MpSigSrv.log` records the refusal by name, which is how you tell the two apart.
>
> **One line gates both halves.** Scanning goes through the same door as signing, so a wrong `ORIGIN` takes out the *Scan* buttons and the pad together. That is worth knowing in reverse too: if scanning and signing are both dead at one desk, suspect this line rather than the hardware.

`[RDD] user` is the account MpSigSrv connects to the dictionary as. **It is not the person who witnesses the signature** — that is the operator code Mp10 Web sends with each capture. Do not put an operator code here: an operator code is not an ADS login and the server answers `7078`.

4.5 Telling MpSigSrv which pad

Two files decide this and they do different jobs.

`C:\Windows\SigPlus.ini`, written by Topaz's installer, says how to reach the pad:

```
[Tablet]
TabletComPort=9
TabletModel=LCD4X5SE
```

The pads at this practice are **serial** devices reached through a USB-to- serial adapter, so they appear as a COM port rather than as a USB device. Find the number in Device Manager under **Ports (COM & LPT)** — the entry whose name or serial number contains `TOPAZ`.

The consent JSON (see below) says how big the screen is, through its `TabletModel` key:

```
{ "TabletModel": 1, ... }
```

`1` means the 320×240 screen; anything else means 240×64.

> **This is not optional and it is not guesswork.** Measured on this practice's pad: `TabletModelNumber()` — the call that asks the device what it is — returned `0` from one program and `58` from another, and `GetLCDSize()` only reports whatever layout was last set, not the physical panel. **The pad does not reliably identify itself.** The value in the JSON is what actually sets the geometry. Get it wrong and the consent is laid out for the wrong screen.

To see what your pad reported, take one signature and read `Topaz.log`:

```
TabletModelNumber()  58   xMax      yMax
TTOPAZSIGNATURE:INITPAD   nTabletModel 1   xMax 240  yMax 64
nTabletModel 1   xMax 320  yMax 240
```

The last line is the geometry in force.

4.6 The consent text

The text, the options and the font all come from `sys_registry`, from the **same row the desktop reads** — so editing it in Admin changes both the desktop and the web at once. For encounters that row is `TOPAZ / ENCOUNTER-SIGNATURE-DISPLAYMESSAGE`.

Its value is either the text itself, or **the path to a file holding it**. This practice uses the file form:

```
\server\share\EncounterSignature-Text.txt
```

and that file holds JSON:

```
{
  "TabletModel": 1,
  "NewPageForOptions": false,
  "NewPageForSignature": false,
  "FontSize": 15,
  "FontName": "Arial",
  "Options": [
    { "id": 1, "Option": "Acepto mis resultados por email…" },
    { "id": 2, "Option": "No acepto el envío…" }
  ],
  "ShowTextOnPad": "Autorizo a … Mi email es: ^email "
}
```

`^email` is replaced with the patient's address at the moment of signing.

> **If the path is a UNC share, the account running MpSigSrv must be able to read it.** A path that works when you are logged in can still fail for a different account. When the file cannot be read, Mp10 Web refuses the capture and names the path — it does not show the patient a file path as though it were a consent, which is what it used to do.

4.7 Check it

Four checks, in this order. Do them on the workstation with the hardware. The last one is only for a desk that scans.

1. Is it running and listening? Open this in the browser:

`http://127.0.0.1:6266/ping`

You should get:

```
{"ok":true,"app":"MpSigSrv","build":"2026-08-11 20:11:26"}
```

Nothing at all means it is not running — start `MpSigSrv.exe -listen` and look at `MpSigSrv.log`.

2. Is it configured the way you think?

`http://127.0.0.1:6266/status`

```
{"ok":true,"app":"MpSigSrv","port":6266,"origins":["https://mp10web.example.org"],
 "service":"not a service - runs in the operator's session",
 "captured":0,"failed":0,"wedged":false}
```

`"origins":[]` is the failure to look for. It means no `[SIGN] ORIGIN`, and every request from a browser will be refused however healthy the rest looks.

3. Does the pad actually work? Without a browser in the way, from a command prompt in the Mp10 program folder:

```
MpSigSrv.exe -enc=EN26-00000123 -operator=RCB
```

Use a real encounter number and a real operator code. The pad should show the consent; sign it and press the pen button. This is the check to run when the browser says something vague, because it takes the browser, the origin allowlist and the network out of the picture and leaves only the pad and the dictionary.

`-operator=` is **required**. It is stored on the signature as the record of who witnessed it, and there is no safe default — the helper refuses without one rather than inventing a value.

MpSigSrv is a windowless program, so it prints nothing to the command prompt. Read `MpSigSrv.log`.

4. Can it see the scanner? Only if this desk is meant to scan:

`http://127.0.0.1:6266/scanners`

```
[{"id":"...","name":"HP CLJM277 Scan Driver","backend":"wia",
  "dpis":[75,100,200,300,600,1200],"colours":["colour","gray","bw"],
  "feeder":true,"duplex":false,"headless":true,
  "cached":true,"probed":"2026-08-15 11:04:22"}]
```

Three answers and they mean different things:

Answer	Meaning
A list with your scanner in it	Working. What it reports is what the browser will offer.
`[]` — an empty list	The helper is fine; Windows sees no scanner. When a scanner is not found.
`No such endpoint: GET /scanners`	This desk's `MpSigSrv.exe` predates scanning. Copy the current one over.

Take the first answer seriously — it is the device's own answer, not a guess. A resolution missing from `dpis` is a resolution that scanner refuses, and the browser will not offer it.

Finally, in Mp10 Web: open an encounter and press **Sign**, and — on a scanning desk — open a patient's **Attachments** and press **Scan...**.

4.8 Updating a workstation later

Stop it first — a running copy holds the exe open and cannot be overwritten:

1. End `MpSigSrv.exe` in Task Manager.
2. Copy the new `MpSigSrv.exe` over the old one.
3. Start it again, or log off and back on if it is started at logon.

The build a workstation is running is reported by `/ping` and `/status`, which is how you confirm the copy actually took.

> **A desk on an old build does not report an error, it reports its age.** Mp10 Web's admin page says *"this workstation's helper predates scanning"* and turns the Scan button off, rather than showing a failure. That is the right behaviour for the operator and a trap for you: **nothing tells anybody to update that desk.** When you roll out a new `MpSigSrv.exe`, the desks you miss stay quietly signature-only.

---

5 Scanning — IT task

Everything in this section is about the scanner half. A desk that only takes signatures can skip it.

5.1 What a scanner needs on the workstation

Nothing that is not already there, in the ordinary case:

1. **The scanner's own Windows driver**, installed by its vendor as usual. If Windows Fax and Scan can see the device, so can MpSigSrv.
2. **`EZTW32.DLL` beside `MpSigSrv.exe`**, and only if the device is reached over TWAIN. It ships with the Mp10 desktop applications.

There is no separate scanning setting. The port, the origin allowlist and the startup entry are the ones already described under *Installing it* — scanning uses the same door.

5.2 WIA and TWAIN, and why both

Windows has two scanner interfaces and devices do not agree on which to offer. MpSigSrv speaks **both**, and neither is a fallback for the other:

- **WIA** is what most modern multifunction devices present.
- **TWAIN** is what a lot of dedicated flatbeds present, and on some hardware it is the *more* capable of the two — it can be told to suppress its own dialog, set resolution directly and report a loaded feeder.

The helper enumerates both, and a device reachable over both is listed once, noting that the other route exists. You do not choose; it uses what the device actually offers.

5.3 How the helper decides what a scanner can do

It asks the device, every time, and it assumes nothing. This matters more than it sounds:

- The **resolutions** offered in the browser are the exact list the device reports. A device offering `{75, 100, 200, 300, 600, 1200}` will *fail* on anything else, several seconds into a transfer, so the browser never offers anything else.
- The **colour modes** are the ones the device accepts.
- **Feeder and duplex** are reported, not assumed. The *Source* choice only appears when the device says it has a feeder.

Enumeration — "what is plugged in right now" — runs on **every** check, which is what notices a scanner unplugged this morning. The expensive part, connecting to each device and reading its full capability list, is cached. That is what `MpScanners.ini` holds.

5.4 The cache, and Re-detect

`MpScanners.ini` sits beside `MpSigSrv.exe`, one section per device, written by the helper. It is **deliberately not in the dictionary**: what a scanner can do is a fact about *that desk*, and putting it in shared storage would let one desk's scanner define another's.

A cached record is used only while all three of these still match:

Keyed on	So it re-probes when
the device's id	a different scanner is attached
the driver version	the vendor's driver is updated
the MpSigSrv build	you deploy a new helper — a newer build may ask better questions

That covers the ordinary cases with no intervention. For the ones it cannot catch — a device replaced by another with the *same* identity, a driver misbehaving, capabilities that simply look wrong — Mp10 Web's admin page has a **Scanners** card with a **Re-detect** button. It forgets every cached record so the next check reads the hardware from scratch.

The same card is where you can see, per device, what the helper thinks it can do and **when it was probed**. A capability record can be weeks old and still be current, which is only obvious when the date is shown.

Deleting `MpScanners.ini` by hand does the same thing as *Re-detect*. It is written from scratch when needed, and nothing else reads it.

5.5 A scanner that owns its own settings

Some TWAIN devices will not let their own settings dialog be suppressed. For those, the browser **hides the resolution and colour controls** and says so:

> *This scanner asks for its own settings when the scan starts, so resolution and colour are chosen there rather than here.*

That is correct behaviour, not a fault. Offering controls the driver is going to ignore would be worse than offering none. The scan still works; the dialog appears **on that workstation's screen**, and somebody has to be sitting there.

5.6 What a scanned page becomes

No page reaches Mp10 Web the way the scanner produced it. Every one goes through the same funnel in the helper, whichever backend acquired it:

```
acquire  ->  scale to 1600 px on the long edge  ->  JPEG, quality 75
         ->  if still too large, step the quality down and shrink again
```

This is not an optimisation; it is the only way a scan fits. One page at 300 DPI in colour — an entirely reasonable choice for a clinical document — came off a real scanner as a **25.6 MB TIFF**, against Mp10 Web's 4 MB ceiling. The helper turned it into **351 KB** in under a fifth of a second.

Two consequences worth knowing before somebody reports them as bugs:

- **A higher DPI does not give a bigger stored file.** It gives a better-sampled one that is then scaled to the same long edge. Beyond a point you are paying scanning time for nothing.
- **The browser shows what was done** — the size that came off the glass, the size stored, and the quality used — underneath the last page. That line is there precisely so an operator who chose 1200 DPI can see why the result is 350 KB.

Several pages become **one PDF**, written by the helper itself. A single page stays a JPEG. The limits, none of which a desk normally meets: **50 pages** per document, **2 MB** per page, **4 MB** for the finished document, and an abandoned scan is discarded after **10 minutes**.

5.7 Cropping a card

The *Scan* button beside the patient ID card and the insurance card scans one page and **crops it to the card**, matching what Patients10 has always done.

The crop is done in the helper, before the page is scaled, which is the whole reason it is worth doing there: a card cropped first and scaled second keeps about **46% more detail on a policy number** than the reverse, for the same stored bytes.

It finds the card by looking for what is darker than the background. **Every way that can fail ends in the whole page being stored, never in an error**: a blank sheet, a lid left open, a crop that comes out absurdly small, or a page it cannot read the pixels of. So the failure a site will actually report is *"the crop*

did nothing" — and the usual cause is a **dark scanner lid, or one left open**, which makes the whole page look like content.

Note also that it crops to whatever is on the glass. A card scanned with a sheet of paper beside it crops to both.

5.8 When a scanner is not found

`http://127.0.0.1:6266/scanners` answering `[]` means the helper is healthy and Windows reports no device. In order:

1. **Is it on, connected, and awake?** A device asleep on USB often enumerates as absent.
2. **Can anything else see it?** Windows Fax and Scan, or the vendor's own utility. If they cannot, this is a driver problem and MpSigSrv is not involved.
3. **Is it a TWAIN-only device with no `EZTW32.DLL` beside the exe?** Then the TWAIN half is simply switched off and reports nothing. Copy the file in and restart the helper.
4. **Press *Re-detect*** on the admin page, in case a stale cached record is in the way.

`MpSigSrv.log` records each probe and what it found.

5.9 What has never been tested

Recorded because meeting one of these unexpectedly is worse than reading about it here. All of the scanning work was proven against **one** device — an HP Color LaserJet MFP M277 — and these paths went untested:

- **A native vendor TWAIN driver.** Every TWAIN test went through Microsoft's WIA-to-TWAIN shim. That proves the plumbing, not how a vendor's own driver behaves.
- **A loaded document feeder.** "Is there another page" comes from the device's own status and has only ever been observed reporting *no*, because every test scan was off the glass.
- **Duplex.** The test device reports none.
- **The quality step-down.** Nothing that scanner produced was large enough to need it.

If a site meets one of these, the log and the browser's per-page line are what to send back.

---

6 Notes for the curious

6.1 Is it safe to have a program listening on the workstation?

It binds **127.0.0.1** only — nothing off that machine can reach it, and no firewall rule is needed. On top of that it refuses any browser request whose origin is not in `[SIGN] ORIGIN`. Because a JSON POST is a *preflighted* cross-origin request, a page from any other site is stopped by the browser before its request is ever sent.

It does not check the Mp10 Web login token, and Mp10 Web deliberately does not send one — the helper has no use for it, and putting a session credential on that wire would only widen what a mistake could leak.

This does not defend against other software already running as that user on that PC. Neither does Topaz's own SigWeb, and that is the same trade the practice already accepted for signature capture.

Scanning was added behind the **same** door rather than a second one, on the same reasoning: the threat model did not change — software already running as that user can drive the scanner without going through us — and a second access control would only be a second thing to get wrong.

6.2 Why does the operator code travel with each capture?

`patientsignatures.operator` records **who witnessed the signature**. The desktop gets that free, because the person signing in to the desktop *is* the dictionary user. MpSigSrv connects as a service account, so if it used the connection's identity every signature would be attributed to `AutoTasks`. That happened once, in testing, which is why the operator is now sent explicitly and a capture without one is refused.

6.3 What happens if a capture hangs

The helper answers `/ping` and `/status` throughout, because HTTP is handled on a separate thread from the pad. A capture that nobody finishes is abandoned after 300 seconds — long, deliberately, because that clock is measuring a patient reading a consent, not a computer doing work.

6.4 Why a scan makes the desk feel slow

The pad, the scanner and every other piece of hardware are driven from **one thread**, and only one job runs at a time. A page takes several seconds, so during a scan that desk's helper answers nothing else. This is fine for one operator at one desk, which is the only thing it is ever asked to be, but it is worth knowing rather than discovering.

6.5 Where the PDF comes from

Mp10 has a PDF library, and this program does not use it. Driven from a scan, it **crashed the helper** — an access violation inside the third-party DLL, before a single page had been added. That is the worst failure this design can have, because the same program is holding the signature pad, and a crash of that kind cannot be caught and recovered from.

So MpSigSrv writes the PDF itself, in about ninety lines. A JPEG is already a legal PDF image stream, so each page goes in exactly as it came out of the scanner — nothing is decoded, re-encoded, or lost. The result was checked against an independent PDF reader rather than only against ours.

6.6 Where scanned pages live while a scan is in progress

In the user's temporary folder, named `mpscan…`, and deleted when the document is attached or cancelled. **They are clinical documents**, so there is a backstop: if the helper is ever killed mid-scan, the pages it was holding are orphaned, and the next start sweeps away any `mpscan` file more than an hour old.

6.7 Does an https site really reach a http://127.0.0.1 helper?

Yes. Browsers treat loopback as a trustworthy origin and exempt it from mixed-content blocking. This is the same thing MpPrintSrv relies on.